

Building a Scalable and Reliable Forum App with Test-Driven Development

Introduction

This essay documents the development of a scalable and reliable forum web application as part of the ESRS project module. The project involved building a Test-Driven Forum App for managing communities, utilizing a CI/CD approach and prioritizing reliability and scalability through proper software architecture and caching.

This document will discuss the team's abstracted Engineering Process, outlining the steps involved and the reasoning behind them. It will include reflections on the successes and challenges encountered, as well as lessons learned.

Further, details of the implementation will be provided, including the tools and processes used for operations such as Trello for task management, GitHub Projects for issue tracking, and Postman for sharing APIs.

The development process, which followed a CI/CD approach, will be discussed in detail, including the GitHub branching flow and the handling of the Integration Process.

Moreover, the document will cover the implementation and architecture of the system, including technologies like React, Django, Redis, and PostgreSQL. It will explain how scalability and reliability were designed for, incorporating concepts such as MVC framework, REST API, authorization system, the "Don't Repeat Yourself" principle, and the Repository Design Pattern. The team's experience with these tools and the resulting outcomes will also be discussed.

The final section is our Sprint Development Blog, which includes our detailed task and progress records throughout the project.

Methodology Overview: Scrum

We used Scrum as the methodology for the project. We needed a lightweight iterative procedure that provided flexibility for team members and a framework for collaborative progress. We modified Scrum to suit our needs: sometimes we used weekly sprints and sometimes 2-week sprints, depending on the complexity of our tasks and team availability. The role of Product Owner rotated between Erfan and Yergali to distribute responsibilities.

Our backend team included Erfan and Hugo, while Yergali and Srinivasan worked on the frontend, and Nareshkumar led documentation.

While we faced challenges with Scrum, such as overhead from meetings and maintaining a development backlog, these structures ultimately brought clarity, collaboration, and stability to our workflow. Software development often deals with ill-formed problems, and methodologies like Scrum provide necessary constraints to keep the team focused and aligned.

Sprint planning, stand-ups, and retrospectives helped us refine our direction and maintain progress, even when the work was complex or ambiguous. The key difficulty lies in consistently adhering to Scrum principles, especially for developers who naturally focus more on coding than on process. However, we found that trusting the methodology led to better results.

Planning & Requirements Analysis

Our task was to build a reliable and scalable application. We chose to develop an internet forum because it required multiple interacting user roles and posed interesting challenges related to performance, reliability, and system architecture.

We began by analyzing existing forum platforms, such as try-phpbb.com and discuss.flarum.org discourse.org, including those that offered admin panel access. This helped us define the scope of our system and informed our implementation choices.

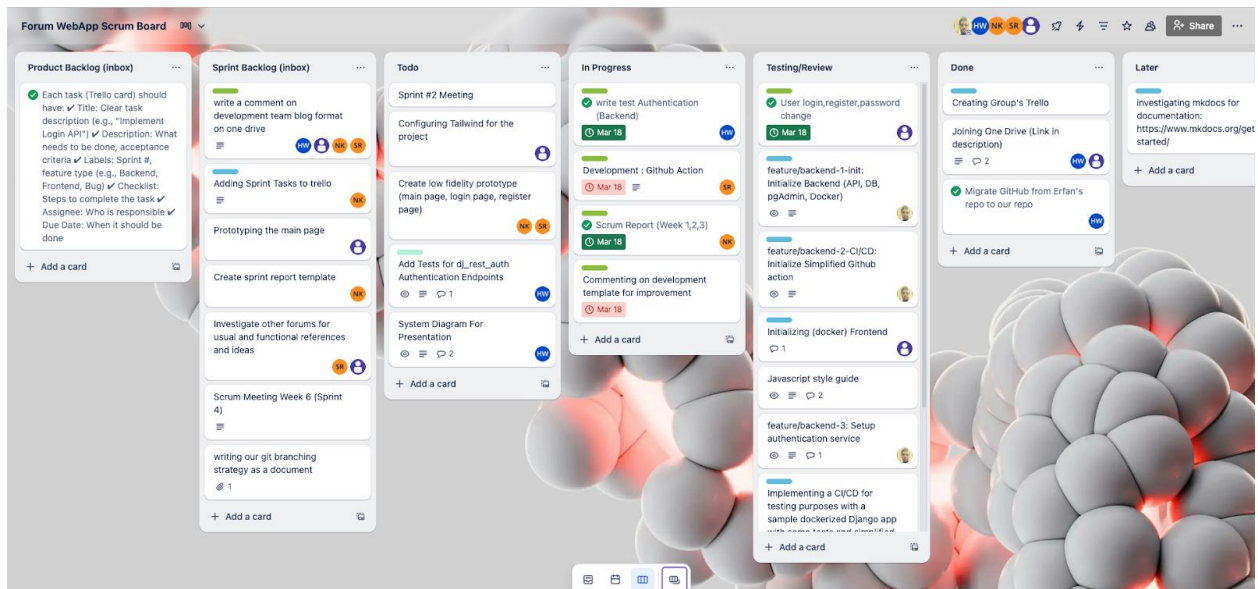
We wrote a detailed set of user stories for our functional requirements, ensuring they captured the needs of Guests, Users, Moderators, and Admins. While this planning phase resembled a waterfall model, we continued to iterate during development, reassessing and improving architectural decisions, tool selection, and design strategies.

For example, although our initial specification included only three user roles, we explored various authentication solutions (e.g., AWS Cognito, Google Identity, Kong API Gateway) to

determine the most scalable and secure setup. Similarly, our choice of PostgreSQL and consideration of domain-driven design principles helped ensure modularity and future scalability.

Collaboration

In order to implement our Scrum methodology backlogs, we used **Trello**. Each sprint's tasks were organized into columns representing their status. Before starting a task, each developer created a Git branch named according to the related Trello task and documented the corresponding commit with a descriptive message. This process supported traceability between the codebase and planned tasks. However, Trello lacked built-in support for managing iterations, and we encountered inconsistencies in issue numbering, which affected our ability to track tasks reliably. As the backlog grew, this became harder to manage.



○ feature/backend-1-init: Initialize Backend (API, DB, pgAdmin, Docker)

 Dates Attachment Checklist

Members

Labels



+

Sprint 1

+

 Description

Edit

Commit message

chore(backend-1): initialize backend with API, DB, and Docker

- Initialized Django REST API with Django REST Framework
- Configured PostgreSQL in docker-compose.yml
- Added pgAdmin for database management
- Created Dockerfile for containerized development
- Closes #1

Branch Name

feature/backend-1-init

We eventually transitioned to GitHub Projects, which offered integrated issue numbering and better linkage between issues and pull requests, streamlining our workflow.

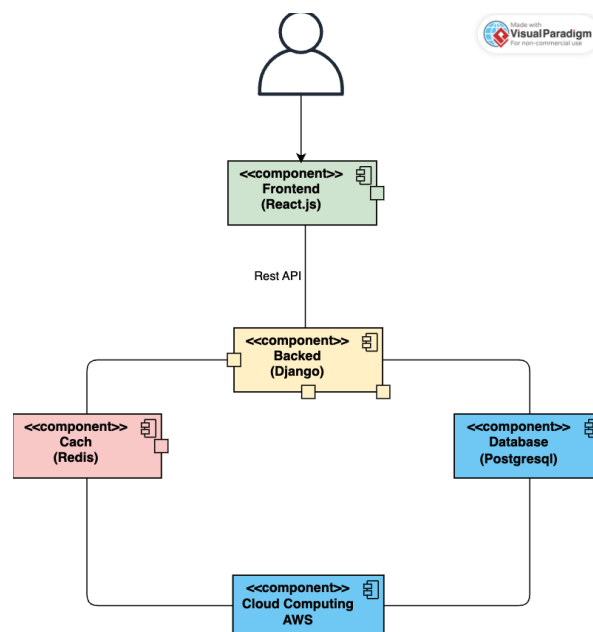
Title	Status	Assignees	Size	Estimate	Linked pull requests	Iteration
P0 (4) (Estimate: 0)						
1 [chore-03] Deployment and Container Orchestration #17	In progress		-	-		Sprint 7
2 [backend-05] Forum main functionalities #1	Done	erfanghoreishi	-	-		Sprint 4
3 [backend-07] Communication System #4	Done	erfanghoreishi	-	-		Sprint 5
4 [chore-02] update devops #7	Done	erfanghoreishi	-	-		Sprint 6
P1 (5) (Estimate: 0)						
5 [feature-11] add more test for authentication #12	In progress		-	-		Sprint 7
6 [feature-12] Forum API improvements for category, topic and tags #2	Done	erfanghoreishi and...	-	-		Sprint 7
7 [bugfix-01] Forum main functionalities enhancements #3	Done	erfanghoreishi	-	-		Sprint 7
8 [backend-09] Caching System #6	Done		-	-		Sprint 6
9 [pressure testing using locust #18			-	-		

For collaborating on our APIs, we used Postman. It allowed us to design, document, and test REST APIs in a shared environment. By sharing example requests and responses across the team, Postman helped ensure consistency between frontend and backend expectations, improved integration accuracy, and reduced miscommunication.

```
{
  "id": 1,
  "username": "testuser",
  "email": "test@example.com",
  "groups": [
    {
      "id": 9,
      "name": "CustomUserChangeGroup"
    }
  ],
  "permissions": [
    {
      "id": 22,
      "codename": "change_customuser",
      "name": "Can change user",
      "content_type": 6
    }
  ]
}
```

Design & Architecture

We employed the MVC (Model-View-Controller) design pattern to enhance the modularity of our system. For the backend, we utilized the Django REST Framework, and for the frontend, we used Next.js, a framework for React.js. We established communication between system components using REST APIs and implemented JWT (JSON Web Tokens) for efficient authentication. Our entire application was containerized with Docker and deployed to our AWS server using a GitHub Action for CI/CD.



Test-Driven Development

To improve the reliability of our project, we adopted **Test-Driven Development (TDD)**. For every new backend feature, we began by writing a test before implementing any functional code. Our approach followed the standard red-green-refactor cycle, ensuring that code was

only written to satisfy failing tests. We avoided writing any implementation code that wasn't first defined by a test case.

TDD significantly enhanced the reliability of our system by maintaining high test coverage and encouraged scalability and modularity. Since developers had to design systems with tests in mind, it naturally led to cleaner, decoupled components and more extensible code structures.

That said, adopting TDD came with challenges. It was especially difficult at the beginning, due to a steep learning curve and the overhead of setting up the proper testing environment. On average, TDD nearly doubled our development time for each feature. However, these initial costs were offset by long-term benefits—particularly as the codebase grew. With increasing complexity and interdependencies, TDD gave us the confidence to refactor and extend our system without fear of introducing undetected bugs.

In addition to reducing debugging time, TDD also served as **living documentation**. Other developers could read our test cases to understand what each part of the system was expected to do, improving both collaboration and maintainability.

```
~ TERMINAL
forum/tests/test_topic.py::test_topic_list_includes_pinned_flag PASSED [ 81%]
forum/tests/test_topic.py::test_topic_list_contains_closed PASSED [ 82%]
forum/tests/test_topic.py::test_topic_list_contains_tags_and_category PASSED [ 83%]
forum/tests/test_topic.py::test_topic_list_not_contain_posts PASSED [ 84%]
forum/tests/test_topic.py::test_guest_can_view_topic_detail PASSED [ 86%]
forum/tests/test_topic.py::test_topic_detail_contains_posts PASSED [ 87%]
forum/tests/test_topic.py::test_topic_detail_contains_tags_and_category PASSED [ 88%]
forum/tests/test_topic.py::test_user_automatically_gets_topic_perm PASSED [ 89%]
forum/tests/test_topic.py::test_user_with_perm_can_change_topic_category PASSED [ 90%]
forum/tests/test_topic.py::test_user_with_perm_can_close_a_topic PASSED [ 91%]
forum/tests/test_topic.py::test_tags_automatically_added_to_topic_on_creation PASSED [ 93%]
forum/tests/test_topic.py::test_topic_last_activity_updates_on_post PASSED [ 94%]
forum/tests/test_topic.py::test_topic_view_tracking_updates_view_count_correctly PASSED [ 95%]
tests/test_cache.py::test_topic_is_cached SKIPPED (Only runs with Redis backend) [ 96%]
communication/tests/test_message.py::test_unauthenticated_user_cant_send_message
WARNING django.request:log.py:241 Unauthorized: /api/communication/messages/ Live log call
PASSED [ 97%]
communication/tests/test_message.py::test_unauthenticated_user_cannot_list_messages
WARNING django.request:log.py:241 Unauthorized: /api/communication/messages/ Live log call
PASSED [ 98%]
tests/test_cache.py::test_cache_key_stored SKIPPED (Only runs with Redis backend) [ 98%]
[100%]

===== warnings summary =====
authentication/tests/authorization/test_access_control.py::test_user_without_permission_cannot_create_group
authentication/tests/test_auth.py::test_login
authentication/tests/test_auth.py::test_password_reset
/Users/erfan/dev/forum-app/backend/venv/lib/python3.11/site-packages/allauth/account/app_settings.py:117: UserWarning: app_settings.AUTHENTICATION_METHOD is deprecated, use: app_settings.LOGIN_METHODS
warnings.warn()

forum/tests/test_models.py::test_topic_with_tags
forum/tests/test_models.py::test_create_five_topics_with_factory
forum/tests/test_models.py::test_create_five_topics_with_factory
forum/tests/test_models.py::test_create_five_topics_with_factory
forum/tests/test_models.py::test_create_five_topics_with_factory
forum/tests/test_models.py::test_create_five_topics_with_factory
/Users/erfan/dev/forum-app/backend/venv/lib/python3.11/site-packages/factory/django.py:187: DeprecationWarning: UserFactory._after_postgeneration will stop saving the instance after postgeneration hooks in the next major release.
If the save call is extraneous, set skip_postgeneration_save=True in the UserFactory.Meta.
To keep saving the instance, move the save call to your postgeneration hooks or override _after_postgeneration.
warnings.warn()

forum/tests/test_models.py::test_topic_with_tags
forum/tests/test_models.py::test_create_five_topics_with_factory
forum/tests/test_models.py::test_create_five_topics_with_factory
forum/tests/test_models.py::test_create_five_topics_with_factory
forum/tests/test_models.py::test_create_five_topics_with_factory
forum/tests/test_models.py::test_create_five_topics_with_factory
/Users/erfan/dev/forum-app/backend/venv/lib/python3.11/site-packages/factory/django.py:182: DeprecationWarning: TopicFactory._after_postgeneration will stop saving the instance after postgeneration hooks in the next major release.
If the save call is extraneous, set skip_postgeneration_save=True in the TopicFactory.Meta.
To keep saving the instance, move the save call to your postgeneration hooks or override _after_postgeneration.
warnings.warn()

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
84 passed, 2 skipped, 15 warnings in 1.09s
(venv) (base) erfan@Erfans-MacBook-Air-3 backend %
```

84 tests successfully passed

Sprint Development Report

The section below presents our Sprint Development Blog, detailing the key tasks and progress made during each sprint.

Sprint 0 (Feb 11–Feb 25): System Specification & Planning

Focus: Initial Setup, System Specification, and Methodology

- Team Formation & Planning:
 - Defined team roles based on strengths: Naresh (Scrum & Docs), Erfan & Hugo (Backend), Ergali & Srinivasan (Frontend)
 - Decided on Scrum + Agile methodology, 1-week sprints, and retrospectives
 - Used Trello and GitHub Projects for backlog and sprint management
- Technical Direction:
 - Selected stack: Django REST (Backend), React.js (Frontend), PostgreSQL, Redis, Docker
 - Chose CI/CD and Kubernetes deployment with GitHub Actions
- Learnings:
 - Importance of flexible planning and lightweight documentation in Agile
 - Created initial user stories and system requirements

Sprint 1 (Feb 25–Mar 4): Design & Initiation

Focus: Project Kickoff and Environment Setup

- Backend Setup (Erfan & Hugo):
 - Initialized Django REST API with PostgreSQL in Docker
 - Configured pgAdmin and created reusable Dockerfiles
 - Created GitHub Actions workflow for CI
- Frontend Setup (Ergali & Srinivasan):
 - Started React project with Docker
 - Created design wireframes on Figma
 - Built basic navigation (Home, Login, Register)
- Team Coordination:

- Set up Sprint Report Template for consistent documentation (by Naresh)
- Identified carryover task: GitHub staging environment (blocked by lack of server)

Sprint 2 (Mar 5–Mar 10): Authentication System

Focus: User Access and Token Security

- What We Did:
 - Backend: Django token-based auth (login, registration, password reset)
 - Frontend: Login, Register, and Password Change forms with error validation
 - Testing: Hugo tested edge cases for invalid tokens and login flows
- Challenges:
 - Syncing token auth across frontend and backend
 - Limited time for endpoint testing (carryover to next sprint)
- Retrospective:
 - Token auth was more complex than expected but now reusable
 - Need earlier test coverage for endpoints

Sprint 3 (Mar 11–Mar 24): Authorization System

Focus: Roles, Permissions, Group Control

- What We Did:
 - Implemented roles: User, Moderator, Admin
 - Added group-based permission system
 - Integrated group-based access checks in the backend
- Carryover:
 - Forgot to document assigned permissions (added to backlog)
- Learnings:
 - Permission granularity improves maintainability
 - Role separation helps in UI tailoring

Week 4 (Mar 25–Apr 7): Forum Core Features

Focus: Categories, Topics, Threads, and Posts

- Backend:
 - Created APIs for categories, threads, posts
 - Linked categories to threads and topics
- Frontend:
 - Dynamic category selection UI
 - Developed forum layout inspired by Reddit & StackOverflow
- Retrospective:
 - UI/UX improved by referencing real-world platforms
 - Redis caching helped speed up load times

Sprint 5 (Apr 8–Apr 22): Messaging & Final Features

Focus: Direct Messaging, Tagging, Notifications

- Features Added:
 - Private messaging between users
 - Tagging system in threads
 - Real-time notifications
- Bug Fixes & Optimization:
 - Polished UI for responsiveness
 - Load testing with Locust
 - Integrated PR review discipline

Sprint 6 (Apr 22–May 1): Finalization, Testing & Deployment

Focus: Final Bug Fixes and System Hardening

- Testing:
 - Used Pytest and Locust for backend
 - Added missing frontend validations
 - Achieved high test coverage, focused on regressions
- CI/CD Pipeline (Final Form):

- Push → Feature Branch: Test/Lint
 - PR → Develop: Build and Staging Deploy
 - Merge to Main → Production Deploy
-
- Tools Used:
 - Docker, GitHub Actions, Kubernetes, Redis, pgAdmin

Sprint Summary Table

Sprint	Dates	Focus	Key Outcomes
0	Feb 11–25	Planning & Spec	System requirements, Stack, TDD research
1	Feb 25–Mar 4	Setup	Init Frontend/Backend, Docker, CI/CD
2	Mar 5–10	Auth System	Token login, validation
3	Mar 11–24	Roles & Permissions	Group-based access
4	Mar 25–Apr 7	Forum Features	Categories, Threads, Posts
5	Apr 8–22	Messaging & Polish	Messaging, Tags, UI enhancements
6	Apr 22–May 1	Testing & Deploy	CI/CD pipelines Improvement, Load testing

Conclusion

This project has been a robust exercise in agile collaboration, system design, CI/CD deployment, and team coordination. We emerged with a scalable, well-tested forum system that aligns with both technical goals and user needs.